



The set covering problem applied to optimisation of gas detectors in chemical process plants

Sávio S.V. Vianna*

UNICAMP - University of Campinas. Cidade Universitária "Zeferino Vaz" Av. Albert Einstein, 500 - CEP 13083-852 Campinas SP Brazil

ARTICLE INFO

Article history:

Received 7 May 2018

Revised 28 September 2018

Accepted 9 November 2018

Available online 22 November 2018

Keywords:

Set covering problem

Graph

Dominating set problem

Gas detectors

CFD

ABSTRACT

An approach to optimise the number and location of gas detectors was developed based on the colour pattern of the graph of the set covering problem (SCP). The optimisation problem is combined to Computational Fluid Dynamics (CFD) data and the discrete optimisation problem is solved using Balas algorithm in the framework of a developed Fortran code. Every computational cell is regarded as a node of a graph where the links are the common boards shared by the neighbouring cells. The graph is read into the code via a dedicated algorithm for generation of the set of constraints. The characteristic length that determines the distance among the nodes of the graph is obtained from the set of CFD simulations or any established criterion. This work is based on the discovery of the colour pattern observed in the adjacency matrix that represents the set of constraints of the optimisation formulation and also on the limited set of gas dispersion CFD simulations to ensure full coverage of the area. Two cases (covering problem and p-median problem) are used in the validation of the developed code. Results are compared with the results obtained using CPLEX. Additional tests are considered ranging from simple 2D cases with 4 nodes up to 75 nodes. Two real engineering cases are considered and the efficiency of the method is discussed based on the volume of the cloud detected and the time to detection. Results show an efficient method for optimisation of gas detectors able to detect flammable clouds as small as a few cubic meters within 10 seconds.

© 2018 Elsevier Ltd. All rights reserved.

1. Introduction

The set covering problem has applications in various branches of engineering field. It is a well defined mathematical problem that can be applied in logistics, scheduling, manufacturing, service planning, airlines crew assignment and chemical process plants (Corbetta et al., 2016), (Turkay and Grossmann, 1996), (Van den Heever and E., 1999).

More recently it has been applied on the modelling of water distribution network. The idea behind the modelling was to optimise the amount of sensors to monitor accidental contamination (Palletia V. et al., 2016). There had been applications on location of sensors combined to reliability engineering where the representation of the system by a digraph (DG) was modelled as a set covering problem (Bhushan and Rengaswamy, 2000). In general, engineering problems that can be represented by digraph can be mathematically modelled by the set covering problem.

One of the first attempts to model the placement of gas detectors combining mathematical programming and computational

fluid dynamics considered a 2D layout of the area to be protected and the set covering problem (SCP), as proposed by Vianna and Ferreira (Vianna and Ferreira Filho, 2004), (Vianna, 2004). The CFD (Computational Fluid Dynamics) simulations were performed using FLACS (Flame Acceleration Simulator) and the optimisation problem solved using the code developed for 0–1 binary optimisation problems. (Vianna, 2004). The work conducted by Vianna and Ferreira (Vianna and Ferreira Filho, 2004), however, only considered the two dimensional domain in the optimisation mesh, although the gas dispersion simulations were conducted using computational fluid dynamics. The approach was not able to handle complex 3D geometry and the set of constraints required manual input.

More recently, Legga et al. (Legga et al., 2012) combined stochastic formulation with FLACS simulations in order to calculate hundreds of gas dispersion scenarios which were used in the optimisation approach to minimize the time to detection. The modelling suggested by Legga et al. (Legga et al., 2012) relies on the number of dispersion scenarios and it does require a significant number of CFD simulations in order to have a candidate for placement of the gas detector. Benavides Serrano et al. (Benavides-Serrano et al., 2015) investigated the gas detector placement approaches and they suggest the incorporation of additional costs in the risk function.

* Corresponding author. Fax: +55 19 3521 3910.

E-mail addresses: svianna@unicamp.br, savio.vianna@cantab.net

For all approaches investigated the dependency on the number of gas dispersion scenarios can significantly increase the cost of the analysis. Legga et al. (Legg et al., 2013) also combined the conditional risk value with the stochastic formulation of the optimisation problem.

The current work investigates the application of the Set Covering Problem (SCP) in the optimisation of gas detectors in chemical process plants. Due to the intrinsic features and the nature of the SCP formulation it has been showed that the number of CFD simulations can be reduced significantly. It is also possible to optimise the gas detector network based on pre-established covering criterion. One example is discussed based on the procedure suggested by BP under the Fire and Gas Detection Engineering Technical Practice GP 30–85.

A novel algorithm for the adjacency matrix of the graph $G = (V, E)$ (GGA - Graph Generator Algorithm) that represents the area to be protected is also introduced. The algorithm (developed in the current research) reduces the data input effort minimising the required modelling time.

The algorithm proposed here is combined with Balas approach for the set covering problem. The new developed code is tested against results of the literature and the classical p-median problem. The p-median test cases are also computed using CPLEX in order to verify the implementation of the procedure to solve 0–1 optimisation problems. The set covering problem (considering the novel GGA algorithm) is then applied to the problem of optimisation of gas detectors in a chemical process area. The minimal number of detectors and their location are calculated using the algorithm presented in this work. The time to detection is also assessed using computational fluid dynamics in order to verify the efficiency of the gas detector distribution.

The results have shown that the approach is not entirely dependent on gas dispersion simulations. It is also shown that the procedure discussed here guarantees 100% coverage of the area as per the mathematical nature of the set covering problem.

The paper is organised as follows. In the next section the set covering problem is introduced. The following section presents the graph generator algorithm proposed in the current work. The efficiency of the algorithm is discussed and a series of covering problems for different instances is used to verify the implementation of the solver. The implementation of the algorithm for the optimisation of the 0–1 integer problem is validated using the p-median problem. The formulation is then applied to a real engineering case where a process area of 576 m² is considered. The optimisation of the number of gas detectors and its respective location is discussed. The results show that clouds as small as 5 m³ are detected within seconds. In the last section, the conclusions are drawn.

2. Covering problems

The problem discussed here is naturally formulated in terms of sets.

Let

$$M = \{1, 2, \dots, m\} \quad (1)$$

be a base set.

The set covering is a sub-family that covers all elements of M . These problems are commonly defined as covering problems and may be defined in a uniform manner;

$$\text{Max}Z = \sum_{j=1}^n c_j x_j \quad (2)$$

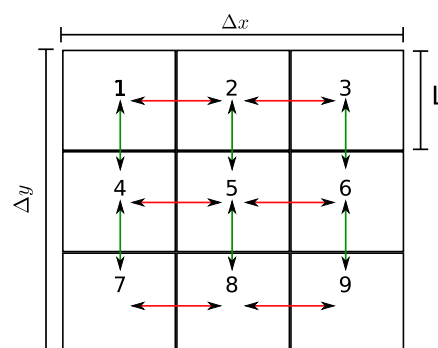


Fig. 1. Representation of the industrial process area of dimension Delta y and Delta x with characteristic length L. The arrows represent the dominance criterion on the basis of the set covering problem (SCP). Every numbered cell represents a sub-area.

subject to:

$$\sum_{j=1}^n a_{i,j} x_j \leq e, i = 1, \dots, m \quad (3)$$

$$x_j = 0, 1, j = 1, 2, \dots, n \quad (4)$$

where e is the m -dimensional vector of values set to unity.

The problem for which the algorithm presented in this work applies is a particular case of the Set Covering Problem, namely the Dominating Covering Problem. The new developed code is able to handle mixed constraints types ($\leq$; $\geq$; $=$).

2.1. Dominating set

The dominating set for a graph $G=(V,E)$ can be used in many applications related to control and dominance of a given area. The problem of finding the minimal number of devices to fully cover a specific area falls well within this class of problems.

Consider the area presented in Fig. 1 where the arrows represent the dominance range of each individual sub-area. The objective is to find in which sub-areas a device should be placed in order to ensure that the whole area is covered.

The problem can be mathematically formulated as the Set Covering Problem;

$$\text{Min}Z = \sum_{j=1}^n c_j x_j \quad (5)$$

subject to:

$$\sum_{j=1}^n a_{i,j} x_j \geq 1, i = 1, \dots, m \quad (6)$$

$$x_j = 0, 1, j = 1, 2, \dots, n \quad (7)$$

where c is the cost vector, x_j is a decision variable and a_{ij} is the element subset of the adjacency matrix of the Graph $G=(V,E)$ that represents the area to be covered and the dominance criterion of each sub-area.

As far as the graph $G=(V,E)$ is concerned, the set V of vertices takes into account all the sub-areas presented in Fig. 1. On the other hand, the set E of edges of the graph accounts for the dominance criterion illustrated by the arrows in Fig. 1. The dominance concept lies on the fact that, if a monitor device is placed on sub-area 5, there is no need to add another monitor device on sub-areas 2, 4, 6 and 8. The same principle applies to all other sub-areas.

The graph $G=(V,E)$ can be represented by its adjacency matrix, where the coefficients of the matrix indicates if a pair of vertices

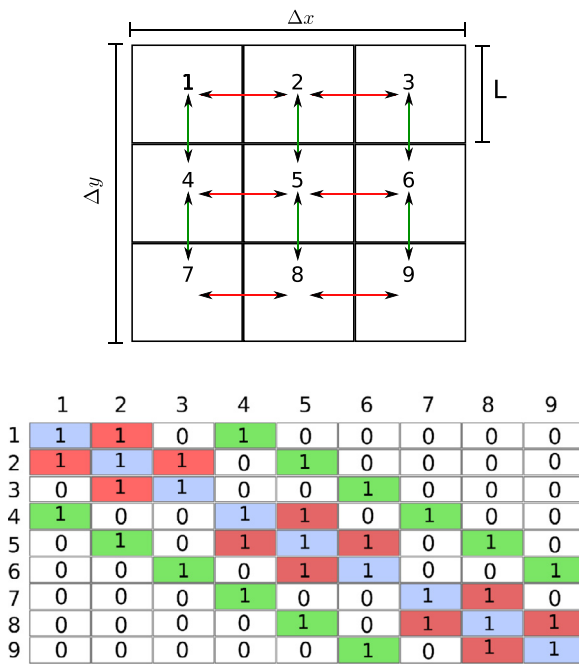


Fig. 2. Adjacency matrix of the graph that represents the target area to be protected. The colour scheme follows the dominance criterion. The main diagonal is set to one as each sub-area protects itself.

are adjacent or not in the graph. Fig. 2 shows the adjacency matrix for the Graph $G=(V,E)$ that represents the area to be covered.

It turns out however, that as the dimensions of the area (Δx and Δy) increases or the characteristic length (L) reduces, the number of sub-areas increases. Consequently, a more complex adjacency matrix is expected.

The level of complexity of the matrix can be prohibitive, leading to additional sources of error during the input process and ultimately yielding a different mathematical model from the desired covering problem.

It is convenient to state that the problem we are investigating in the current work aims to find the sub-areas in Fig. 1 where a monitor device should be placed in order to cover the entire region, represented by Δx and Δy . Additionally, it is desirable to accomplish the covering of the area, by placing monitor devices in a minimal number of sub-areas.

The following section proposes an algorithm that generates the adjacency matrix of the graph $G=(V,E)$ that represents the dominating criterion in the Dominating Set Problem.

3. Graph generator algorithm - GGA

Analysis of Fig. 2 shows that there is a relation between the colours used in the matrix and the arrows presented in the area to be protected. In fact, a closer look in the matrix shows a pattern that follows the dominance criterion represented by the arrows in Fig. 2.

The Graph Generator Algorithm for the Dominating Set Problem combines the colour scheme presented in Fig. 2 according to their appearance in the coefficients of the adjacency matrix.

Following the colour pattern from Fig. 2, the GGA algorithm may be described as follows; Step (2) initialises the problem by inserting the dimensions (Δx and Δy) of the area to be protected and the characteristic length (L) of each cell. Step (3) calculates the number of cells of the matrix. Step (5) assigns 0 to all elements of the matrix. Step (6) set the main diagonal elements to 1.0 (blue diagonal in Fig. 1) Step (8) sets the elements of the diagonal above

and below the main diagonal (red in Fig. 1) to one. Step (10) and step (11) make sure the covering constrain is respected by setting all elements in the green diagonal (Fig. 2) to one.

Algorithm 1 Graph Generator Algorithm.

```

1: procedure GGA: Compute  $A = [a_{i,j}]$ 
2: Input  $n_i, n_j$ 
3:  $cells \leftarrow n_i * n_j$ 
4:  $n \leftarrow n_i$ 
5:  $a_{i,j} \leftarrow 0$ 
6: while ( $i=j$ )  $a_{i,j} \leftarrow 1$ 
7:   for all cells do
8:     while ( $j=i+1$ ) and ( $i=j+1$ )  $a_{i,j} \leftarrow 1$  if ( $j = k*n_i + 1$  or  $j = k*n_i$ )
9:   otherwise  $a_{i,j} \leftarrow 0$ 
10:   $a(k, k-n_i) \leftarrow 1.0$  for  $k = (n+1)$  ; cells
11:   $a(k, k+n_i) \leftarrow 1.0$  for  $k = 1$  ; ( $cells-n$ )
12:  end for
13: end procedure

```

4. Computer code implementation

Prior to applying the proposed GGA algorithm, the framework of the optimisation tool was developed and validated.

The code implementation has followed the standard form of the binary linear programming for zero-one problems.

By applying the classical search approach, the nodes which are connected by the branch of the tree have been set to one or zero or free. A new node can be defined by fixing the value of the variable to one (moving forward in the tree) and a node is revisited if the variable is set to zero (moving backward).

In summary the procedure applied here consisted of three main tasks:

- Set the free variable x_{js+1} at value 1
- Solve the sub-problem in the remaining free variables
- Set x_{js+1} at value 0 and repeat the process with $x_{js+1}=0$

In order to keep track of the enumeration process the Balas zero - one additive algorithm has been applied. The algorithm has been coded in Fortran and it serves as the main routine within Optimi. Detailed description of the steps of the algorithm as it has been coded can be found in (Balas, 1965). The graphic visualisation of the results is obtained using Paraview.

5. Results

The developed code was applied to problems with different levels of complexity. The verification process was divided in two steps.

The first part verified the implementation of the Balas algorithm to solve 0–1 integer problems. Two problems have been considered, namely the trivial test problem and the p-median test problem. It is important to mention that at this stage the GGA algorithm was not considered. In fact, only the coding of Balas procedure was verified.

The second part of the results validate the GGA algorithm when coupled with the 0–1 optimisation code developed in the current work for the set covering problem. A case study was also considered where a gas detector optimisation procedure is suggested.

5.1. Verification - Trivial test problem

The standard formulation for the zero-one optimisation problem has been considered as the first test.

Table 1

Problem parameters for the zero-one standard optimisation problem as coded in Optimi. The problem is resolved via the implicit enumeration algorithm.

Input	description
m	number of constrains
n	number of variables
a [1.m,1.n]	array of constraint matrix
b [1.m]	array of right hand side vector
c [1.n]	array of the cost vector

The numerical problem has been extracted from Kowalik Venart (2004) and the mathematical formulation is presented below;

Minimize

$$Z = 10x_1 + 14x_2 + 21x_3 + 42x_4 \quad (8)$$

Subject to:

$$-8x_1 - 11x_2 - 9x_3 - 18x_4 \leq -12 \quad (9)$$

$$-2x_1 - 2x_2 - 7x_3 - 14x_4 \leq -14 \quad (10)$$

$$-9x_1 - 6x_2 - 3x_3 - 6x_4 \leq -10 \quad (11)$$

$$x_1, x_2, x_3, x_4 = 0, 1 \quad (12)$$

Table 1 presents the description of all variables used in the problem formulation.

The calculation procedure is split into three main parts. The data reading, the implicit enumeration and the results post processing. The data reading is performed in accordance with the parameters described in Table 1. For this particular problem the number of variables and the number of constrains are set to be $n = 4$, $m = 3$, respectively.

The constrain matrix $a[1.m, 1.n]$, the right hand side vector $b[1.m]$ and the cost vector $c[1.n]$ are presented below

$$a[1.3, 1.4] = \begin{bmatrix} -8 & -11 & -9 & -18 \\ -2 & -2 & -7 & -14 \\ -9 & -6 & -3 & -6 \end{bmatrix}$$

$$b[1.3] = [-12, -14, -10]$$

$$c[1.4] = [10, 14, 41, 42]$$

The solution obtained is

$$X[1.4] = [1, 0, 0, 1]$$

$$Z = 52$$

Although the problem considered so far is simple, it had been very important along the process of development of the software. This example helped throughout the debugging phase, errors identification as well as on finding the best strategy and architecture during the coding process.

5.2. Verification - The p-median problem

The p-median problem is a location problem with several applications in engineering. The problem is classified as NP-hard what makes the current test case far more complicated than the previous test case considered. The idea behind the p-median problem is to locate p facilities at a location J aiming to minimise the distance

Table 2

Problem parameters for the zero-1 p-median optimisation problem.

Input	description
d	distance between demand node i and site j
i	number of demand nodes
j	number of sites
p	number of facilities
x	decision variable for demand node served by facility
y	decision variable for facility location

Table 3

Results obtained applying Optimi and CPLEX for the p-median problem considering 10 facilities and the distances between the demand nodes and sites as presented in matrix d_{ij}

N° of facilities	1	2	3	4	5	6	7	8	9	10
OPTIMI	79	47	35	26	18	12	8	5	2	0
CPLEX	79	47	35	26	18	12	8	5	2	0

between the demanded node i and the j facility that will attend the demanded node.

$$\text{Min}Z = \sum_{i \in I} \sum_{j \in J} d_{i,j} x_{i,j} \quad (13)$$

subject to:

$$\sum_{j \in J} x_{i,j} \geq 1, i \in I \quad (14)$$

$$x_{i,j} - y_j \leq 0, i \in I, j \in J \quad (15)$$

$$\sum_{j \in J} y_j = p \quad (16)$$

$$x_{i,j} \in \{0, 1\}, i \in I, j \in J \quad (17)$$

$$y_j \in \{0, 1\}, j \in J \quad (18)$$

Table 2 presents the description of all variables used in the p-median problem modelling.

The p-median problem discussed above has also been resolved using CPLEX. The results obtained via CPLEX have been compared with the findings from Optimi. The total number of facilities selected was 10 with the distances between demanded nodes and sites presented below in matrix d_{ij} .

$$d[i, j] = \begin{bmatrix} 0. & 3. & 10. & 14. & 13. & 18. & 15. & 15. & 13. & 6. \\ 3. & 0. & 7. & 11. & 10. & 15. & 12. & 17. & 15. & 8. \\ 10. & 7. & 0. & 4. & 6. & 11. & 8. & 16. & 22. & 15. \\ 14. & 11. & 4. & 0. & 4. & 9. & 6. & 14. & 22. & 15. \\ 13. & 10. & 6. & 4. & 0. & 5. & 2. & 10. & 18. & 11. \\ 18. & 15. & 11. & 9. & 5. & 0. & 3. & 11. & 23. & 16. \\ 15. & 12. & 8. & 6. & 2. & 3. & 0. & 8. & 20. & 13. \\ 15. & 17. & 16. & 14. & 10. & 11. & 8. & 0. & 16. & 9. \\ 13. & 15. & 22. & 22. & 18. & 23. & 20. & 16. & 0. & 7. \\ 6. & 8. & 15. & 15. & 11. & 16. & 13. & 9. & 7. & 0. \end{bmatrix}$$

Table 3 presents the results obtained for the problem. Comparison of the results calculated using Optimi are in accordance with the results obtained using CPLEX.

5.3. Validation - Set covering problem

A set of 9 cases varying the input data from 25 to 14,400 cells was considered. The set of constrains was calculated using the GGA

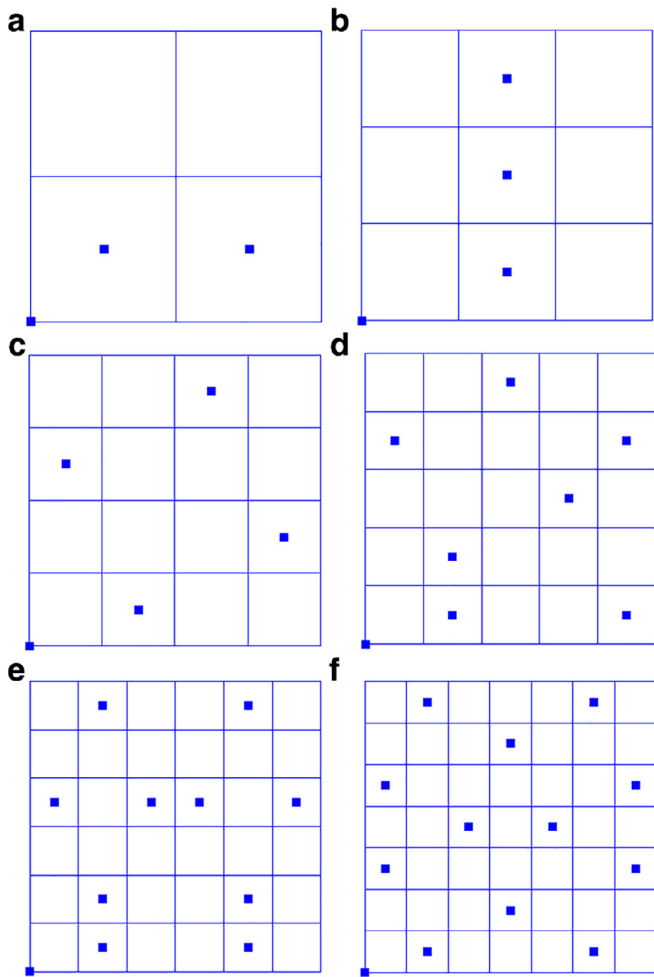


Fig. 3. Optimal solution for the dominating set problem considering (a) 4 sub-areas, (b) 9 sub-areas, (c) 16 sub-areas, (d) 25 sub-areas, (e) 36 sub-areas and (f) 49 sub-areas. The blue dot on the lower left corner of the first cell is not part of the optimal set. It is a reference for the origin of the first cell. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

algorithm. The problem formulation is presented below;

$$\text{Min} Z = \sum_{j=1}^n x_j \quad (19)$$

subject to:

$$\sum_{j=1}^n a_{ij} x_j \leq b_j, i = 1, \dots, m \quad (20)$$

$$x_j = 0, 1, j = 1, 2, \dots, n \quad (21)$$

The x_j (decision variable) represents the sub-areas which are a candidate to receive a “protecting device”. The a_{ij} in the set of constraints are the coefficients of the adjacency matrix of the graph $G=(V,E)$ that represents the area to be covered. The set of constraints ensures that the area is protected by at least one “protecting device” placed at one of the sub-areas. The decision variable x_j receives the value 0 or 1 depending on whether the sub-area is part of the set of the optimal solution or not. The objective function Z must be minimised. It allows for the calculation of the minimal number of sub-areas where the “protecting device” must be placed in order to guarantee 100% of coverage.

Fig. 3 presents the optimal results (blue dots) using the proposed algorithm for 6 different problems. The number of sub-areas

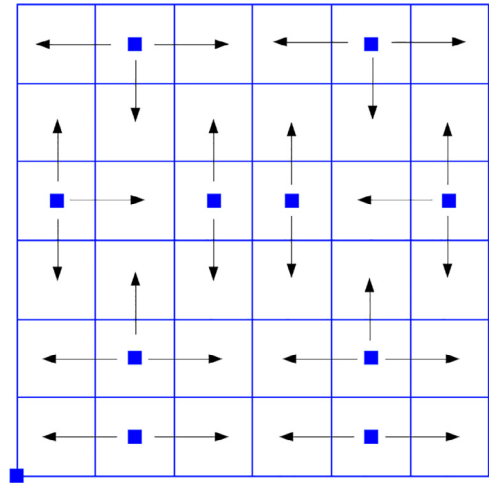


Fig. 4. Optimal solution for the set covering problem using the GGA algorithm. The optimal solution is represented by the blue dots. The set V of the graph $G=(V,E)$ comprises 36 vertices. The optimal subset $S \subset V$, contains 10 elements. The respective dominance is represented by the arrows if accordance with the set of constraints mathematically expressed by Eq. 26. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

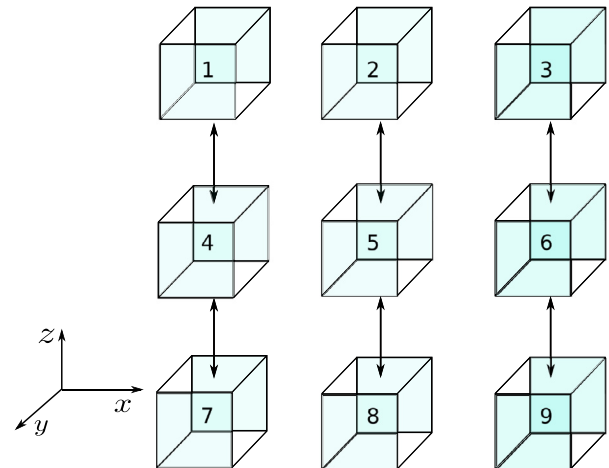


Fig. 5. Vertical dominance in the three dimensional case. In the 3D case, the same sub area 5 previously discussed in the 2D case will also protect the subarea 2 and 8.

varied from 4 to 49. Analysis of Fig. 3 (a) shows that for a total of 4 sub-areas the solution for this optimisation problem is trivial. On the other hand, as the number of sub-areas increases, the level of complexity also increases.

As far as the set up of the constraints equations are concerned it is worth mentioning that an area comprising 49 sub-areas would require 2401 coefficients in the set of constraints. Should the area increase to 100 by 100 cells, 10^4 elements would be necessary during the modelling phase of the problem.

Analysis of Fig. 3 shows that all sub-areas are protected by at least one sub-area tagged with the blue dot. The covering of the sub-areas can be better understood by analysing Fig. 4 where the black arrows indicates the covering range.

The same coverage principle previously discussed applies to the three dimensional case. In the 3D model, every sub-area is also influenced by the sub-areas located above and below as shown in Fig. 5

Fig. 6 shows how the computational time varies with the number of cells. For cubic regions, the number of cells increases to the

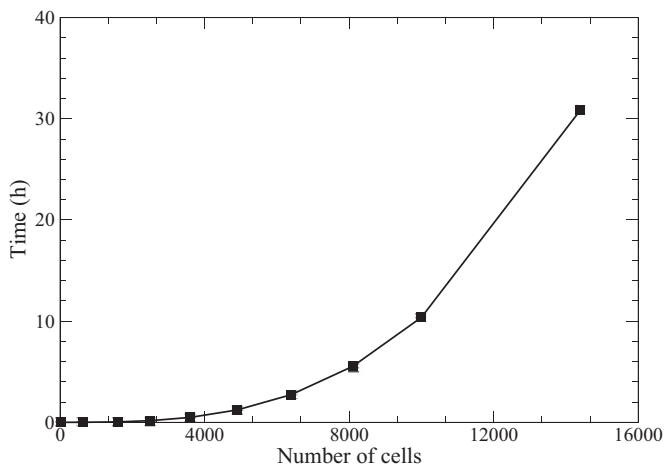


Fig. 6. Computational time versus number of cells. $t(n) = 4.21 * n^{2.98}$, where n is the number of cells.

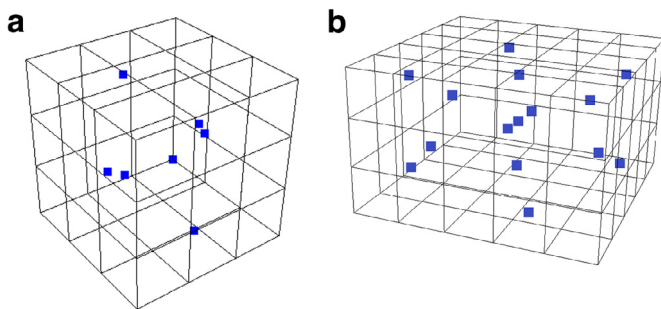


Fig. 7. Optimal solution for the 3D dominating set problem using (a) three layers and 27 cells and using 75 cells (b).

Table 4

Time required to compute the adjacency matrix for increasing number of nodes of the Graph $G=(V,E)$.

thread	Nodes	t (h)	$t/t_{\max}$
1	14,400	30.86	1.0
	10,000	10.37	0.34
	8100	5.53	0.18
	6,400	2.74	0.09
	4900	1.22	0.04
	3600	0.48	0.02
	2500	0.16	0.005
	625	0.04	0.001
	25	0.003	0.00009

power of 3. Based on the expression for $T(n)$, the computational time is also scaled to the power of approximately 3.0.

Fig. 7 shows the optimal solution for the 3D case considering 27 cells and 75 cells. Analysis of Fig. 7 (a) and (b) shows that all nodes (cells) are reached by the optimal set calculated. The optimal set is represented by the dot at the centre of the each cell.

Considering the different levels of complexity, the time required to compute the adjacency matrix can be expressed as ;

$$T(n) = 4.21 * n^{2.98} \quad (22)$$

where n is length of input data. For graphs up to 3600 vertices, the GGA algorithm calculated the respective adjacency matrix within minutes. For the number of vertices varying from 3600 to 10,000 the computing time was found to be within hours, while the computational time required for the graph comprising 14,400 vertices was a little longer than a day. The results are summarised in Table 4.

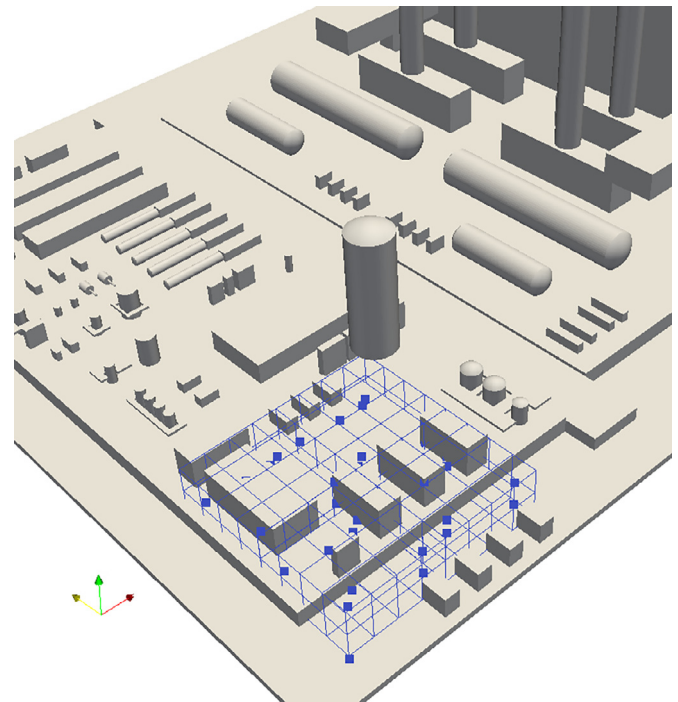


Fig. 8. Output file from Optimi showing the optimisation mesh and the respective spatial location of the gas detectors in the 3D CAD model. An area of 576 m^2 is considered. An overall of 32 point detectors are necessary to cover the full region.

The set of dominance constraints of the set covering problem (formulated in accordance with equations 25, 26 and 27) was calculated using the GGA algorithm and the results were applied in the solution of the covering problem. It has been considered 8 different problems with the number of cells varying from 4 to 75. For all cases the optimal solution was obtained and the dominance constrain has been respected as presented in Fig. 7 for the case with 36 vertices. In Fig. 7, every other cell which is not tagged by the blue mark does not belong to the final optimal solution subset. However, it can be reached (see arrows), by at least, one of the cells where there is a blue mark in it. In fact, it reflects the constrain imposed by Eq. 26. The same dominance constrain applies for those cases considered in the tri-dimensional domain, as shown in Fig. 5.

All calculations were performed using an Intel (R) Xeon (R) processor @ 2.0 GHz and 66Gb of memory.

5.4. Case study

The set covering problem was applied to two real engineering cases.

In the first case the size of the optimisation mesh was set based on the size of the smallest cloud (around 20 m^3) calculated using computational fluid dynamics and reported by Fiates and Vianna (Vianna et al., 2015). For the selected leak location 8 wind directions and 5 leak directions were considered. The wind speed was set as 6.0 m/s and the leak rate considered was 5.0 kg/s.

The second case study did not consider any CFD simulation during the optimisation modelling of the problem. The size of the optimisation mesh was set to 5 x 5 metres in accordance with the Guidance on Practice for Fire and Gas Detection GP 30-85.

5.4.1. Gas dispersion cases combined to the dominating covering problem

The mesh size was calculated in accordance to:

$$L = \sqrt[3]{\text{Min}(V_{i,\%LEL})}, i = 1, \dots, n \quad (23)$$

Table 5

Location of gas detector for the area measuring 12 x 12 x 6 metres. Refer to Fig. 8 (Origin is located at lower left corner).

Detector	x (mm)	y (mm)	z (mm)
1	10,500	1,500	1500
2	4500	4,500	1500
3	1500	10,500	1500
4	10,500	10,500	1500
5	1500	1,500	4500
6	10,500	1,500	4500
7	4500	7,500	4500
8	7500	7,500	4500

Table 6

Set of transient CFD cases used to verify the efficiency of the gas detector location.

CFD Run #	Leak dir.	Wind dir.	Detec. time (s)	Vol. (m3)
1	Down	North	within 5	31.57
2	Horizontal (+x)	East	10	9.02
3	Horizontal (-x)	West	N.A	1.98
4	Horizontal (-y)	North	8	5.32
5	Horizontal (+y)	North	5	5.04

where n is the number of CFD simulations and $V_{i\%LEL}$ is the volume of the cloud at a prescribed concentration. In the study reported by Fiates and Vianna (Vianna et al., 2015), $n = 40$ and the smallest cloud size was found to be 20.59 m^3 .

The mathematical model of the set covering problem comprises the objective function that must be minimised and the set of constraints which ensure the area coverage. The inequalities in the set of constraints ensure that at least one detector will protect the area. The decision variable x_j is 1.0 if it belongs to the optimal solution set and zero otherwise.

$$Z = \sum_{j=1}^n c_j x_j \quad (24)$$

subject to:

$$\sum_{j=1}^n a_{i,j} x_j \geq 1, i = 1, \dots, m \quad (25)$$

$$x_j = \{0, 1\} \quad (26)$$

The characteristic mesh size L was calculated using equation 29. The area to be covered is 12 metres long by 12 metres wide and 6 metres high. The optimal location of the gas detectors is shown in Table 5.

In order to verify the proposed location of gas detectors a set of additional 5 CFD simulations have been performed. The transient simulation of the gas leak have been conducted for the mass, momentum, energy and chemical species equations using the Reynolds Averaged Navier-Stokes (RANS) approach. The turbulence problem was addressed via the Bousinessq approach and the classical $K - \epsilon$ turbulence model was used. The set of CFD simulations is listed in Table 6. A set of monitor points has been placed at the same spatial location of the detectors calculated in the optimisation procedure. Five different leak directions have been considered and three different wind speeds. For all cases the gas cloud has been detected within 10 seconds after the release. One of the cloud has not been detected. It turns out, however, that the volume of the cloud was far too small to be detected with the current set up. The calculated distribution of the detector was able to detect clouds as small as 5 m^3 .

Fig. 9 shows how the gas cloud concentration varies in time. The gas releases starts at 70 seconds. Analysis of Fig. 9 shows that the gas cloud is detected right at the start of the gas leak.

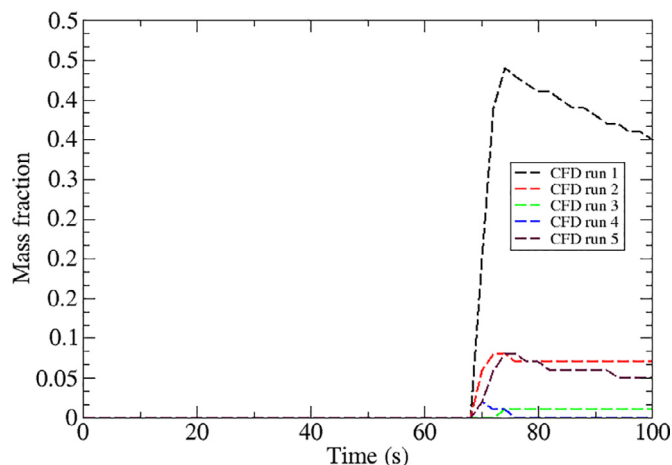


Fig. 9. Gas cloud concentration in the set of optimised gas detectors as function of time. The gas leak starts at 70 seconds. The LFL is 5%. Most of the gas leak scenarios are detected within 10 seconds.

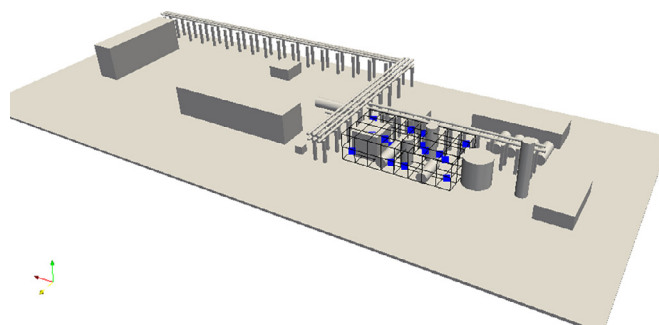


Fig. 10. Optimised distribution of gas detectors based on the criterion suggested by the Fire and Gas Detection Engineering Technical Practice, from British Petroleum GP 30–85.

5.4.2. Coverage criterion combined to the dominating covering problem

This second case study differs slightly from the first case investigated. The optimisation mesh was set in accordance with the Fire and Gas Detection Engineering Technical Practice, from British Petroleum GP 30–85.

In this case, a regular 5 x 5 metres grid was applied in a process area measuring 40 x 20 metres. A total of 16 detectors was calculated as shown in Fig. 10.

6. Concluding remarks

The current work discussed the application of the Set Covering Problem (SCP) in the optimisation of gas detectors in process plants. Gas dispersion CFD data was combined with 0–1 integer programming in order to calculate the best location and the minimum number of gas detectors in facility site.

An algorithm for generation of the adjacency matrix of the Graph $G = (V, E)$ that represents the coverage area of the Set Covering Problem is provided. The algorithm is based on the colour pattern of the dominance criterion. The algorithm can generate the adjacency matrix for 16,000 cells within hours. It has been shown that the computation time (in seconds) varies with the number of cell as $t(n) = 4.21 * n^{2.98}$. The developed method was verified using CPLEX for the p-median problem as well as the covering problem as reported Kowalik Venart (2004).

Combination of the GGA (Graph Generator Algorithm) algorithm and the Balas procedure for 0–1 integer problems led to optimal number of gas detectors in a real chemical process area with re-

duced number of CFD (Computational Fluid Dynamics) gas dispersion simulations. These might drive significant advances in the area of optimisation of monitoring devices, such as gas detectors.

As future work is worth investigating the effect of weighted nodes of the covering graph based on the leak frequency of process areas.

Acknowledgments

This work is supported by FAEPEX - University of Campinas (Project number Proj - 631). Thanks are also due to St Edmunds College, University of Cambridge - UK and The National Centre for High Performance Computing in Sao Paulo, Brazil - CENAPAD-SP.

References

- Balas, E., 1965. An additive algorithm for solving linear programs with zero-one variables. *Oper. Res.* 13, 517–545.
- Benavides-Serrano, A.J., Mannan, M.S., Laird, C.D., 2015. A quantitative assessment on the placement practices of gas detectors in the process industries. *J. Loss Prev. Proc. Ind.* 35, 339–351.
- Bhushan, M., Rengaswamy, R., 2000. Design of sensor location based on the various fault diagnostic observability and reliability criteria. *Comput. Chem. Eng.* 24, 735–741.
- Corbetta, M., Grossmann, I., E., Manentia, F., 2016. Process simulator-based optimization of biorefinery downstream processes under the generalized disjunctive programming framework. *Comput. Chem. Eng.* 88, 73–85.
- Van den Heever, S.A., E., G.I., 1999. Disjunctive multiperiod optimization methods for design and planning of chemical process systems. *Comput. Chem. Eng.* 23, 1075–1095.
- Legg, S.W., Benavides-Serrano, A.J., Sirola, J.D., Watson, J.P., Davis, S.G., Brateteig, A., Laird, C.D., 2012. A stochastic programming approach for gas detector placement using CFD-based dispersion simulations. *Comput. Chem. Eng.* 47, 194–201.
- Legg, S.W., Wang, C., Benavides-Serrano, A.J., Laird, C.D., 2013. Optimal gas detector placement under uncertainty considering conditional-value-at-risk. *J. Loss Prev. Proc. Ind.* 26, 410–417.
- Palletia V., R., Narasimhana, S., Rengaswamy, R., Tejab, R., Murty Bhallamudib, S., 2016. Sensor network design for contaminant detection and identification in water distribution networks. *Comput. Chem. Eng.* 87, 246–256.
- Turkay, M., Grossmann, I.E., 1996. Logic-based MINLP algorithms for the optimal synthesis of process networks. *Comput. Chem. Eng.* 20, 959–978.
- Venart, J.E.S., 2004. Flixborough: the explosion and its aftermath. *Proc. Saf. Env. Prot.* 82, 105–127.
- Vianna, S. S. V. (2004). Optimisation of gas detectors using mathematical programming and computational fluid dynamics. Master thesis, COPPE- UFRJ. Federal University of Rio de Janeiro.
- Vianna, S. S. V., & Ferreira Filho, V. J. (2004). A proposal for optimisation and location of gas detectors using using mathematics programming and computational fluid dynamics - congresso latino iberoamericano de investigacion de operaciones y sistemas. 4–8 October. La Habana, Cuba.
- Vianna, S.S.V., Fiates, J., Simoes, V., 2015. OPTIMI - a Novel Computational Tool for Gas Detectors Optimisation. In: Proceedings of the 18th mary kay o'connor process safety center annual international symposium., pp. 1171–1185.